ECE391
Computer System Engineering

Lecture 9

Prof. Kirill Levchenko and Dong Kai Wang

Slides Adapted from Prof. Kalbarczyk

University of Illinois at Urbana- Champaign

Spring 2024

ECE391 EXAM 1

- **EXAM 1 – February 20 (Tuesday)**
  - Time: 7:00-9:00PM
  - Location: ECEB 1002 / 1015
- **Conflict Exam**
  - Deadline to request conflict exam: Thursday, February 15. Send email to both instructors.
- **Exam 1 Review Session by TAs**
  - Date: Saturday February 17th
  - Time: 3:00 to 5:00pm
  - Location: ECEB 1002
- **No Lecture next Tuesday**

Quick Recap Quiz: Interrupts

- Which of the following generates hardware interrupts?
  - A network packet arrives at NIC.
  - An `int 0x80` instruction is encountered.
  - In a multicore CPU, *core 0* tells *core 3* to park.
  - You plug in the charger to your laptop.
- *(True or False)* A CPU handling an interrupt can be interrupted by a different interrupt.
- What are advantages of polling vs interrupts?
  - Don't forget the overheads.

Recall: Interrupt vs Polling

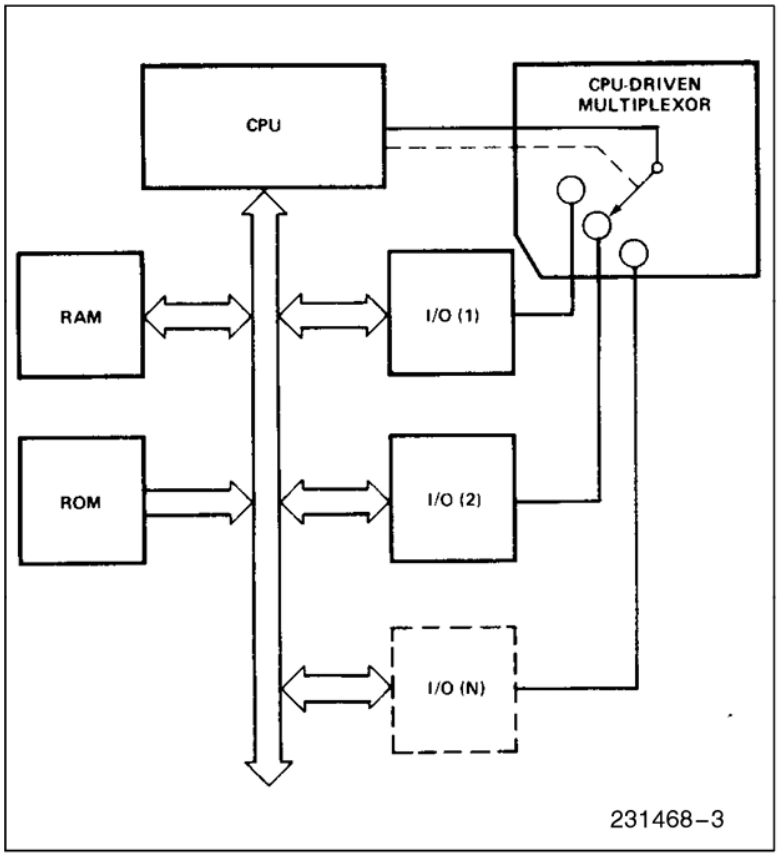


Figure 3a. Polled Method

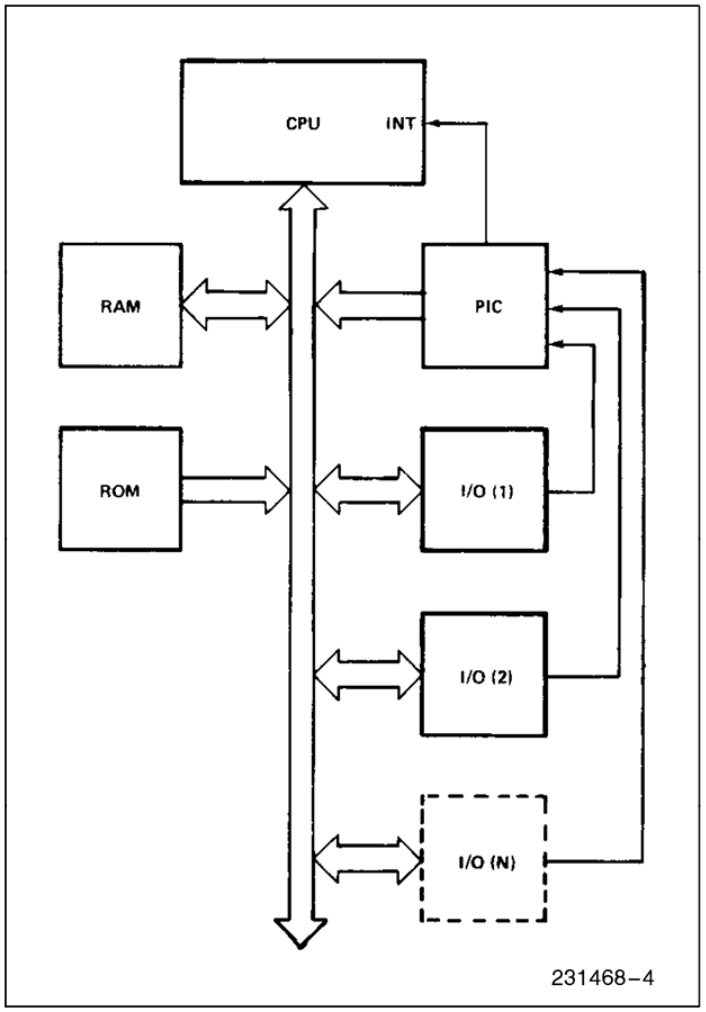
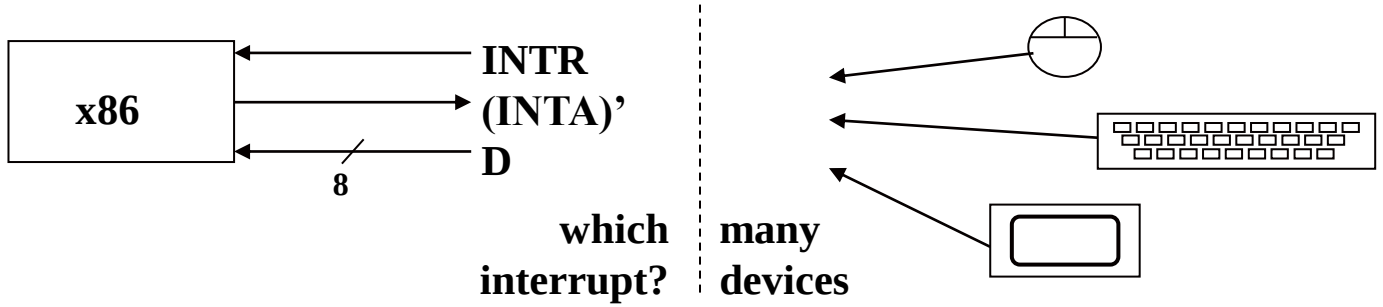


Figure 3b. Interrupt Method

PIC Motivation and Design



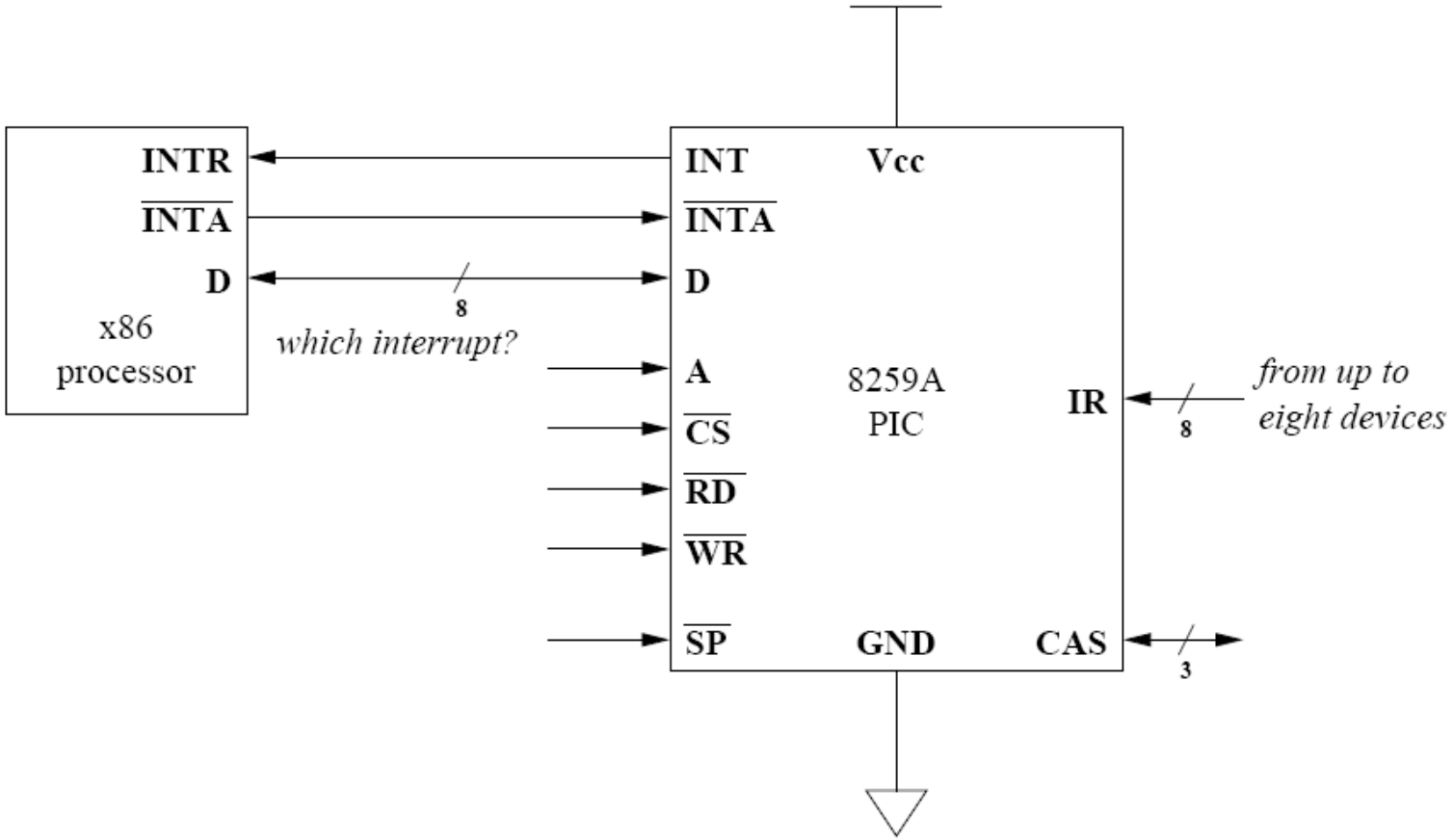
- How do we connect devices to the processor's interrupt input?
- An **OR** gate? Why not?
  - who writes the vector # ?
    - possible to build arbiter, but…
    - what if more than one raised interrupt?
  - extra work for processor to query all devices
    - must execute interrupt code for device that raised interrupt.
    - could have been more than one device.
    - no way to tell with OR gate.

PIC Motivation and Design (cont.)

- not all devices support query.
  - many devices too simplistic to support query.
  - and operations (e.g., reading from port) may not be idempotent.

- nice to have concept of priority and preemption,
  i.e., interrupting an interrupt handler.

8259A
Programmable Interrupt Controller (PIC)



Logical Model of PIC Behavior

- Watch for interrupt signals:
  - from up to eight devices.
  - one interrupt line each.

- Using internal state:
  - track which devices/input lines.
  - are currently in service by processor.
  - i.e., processor is executing interrupt handler for that interrupt.

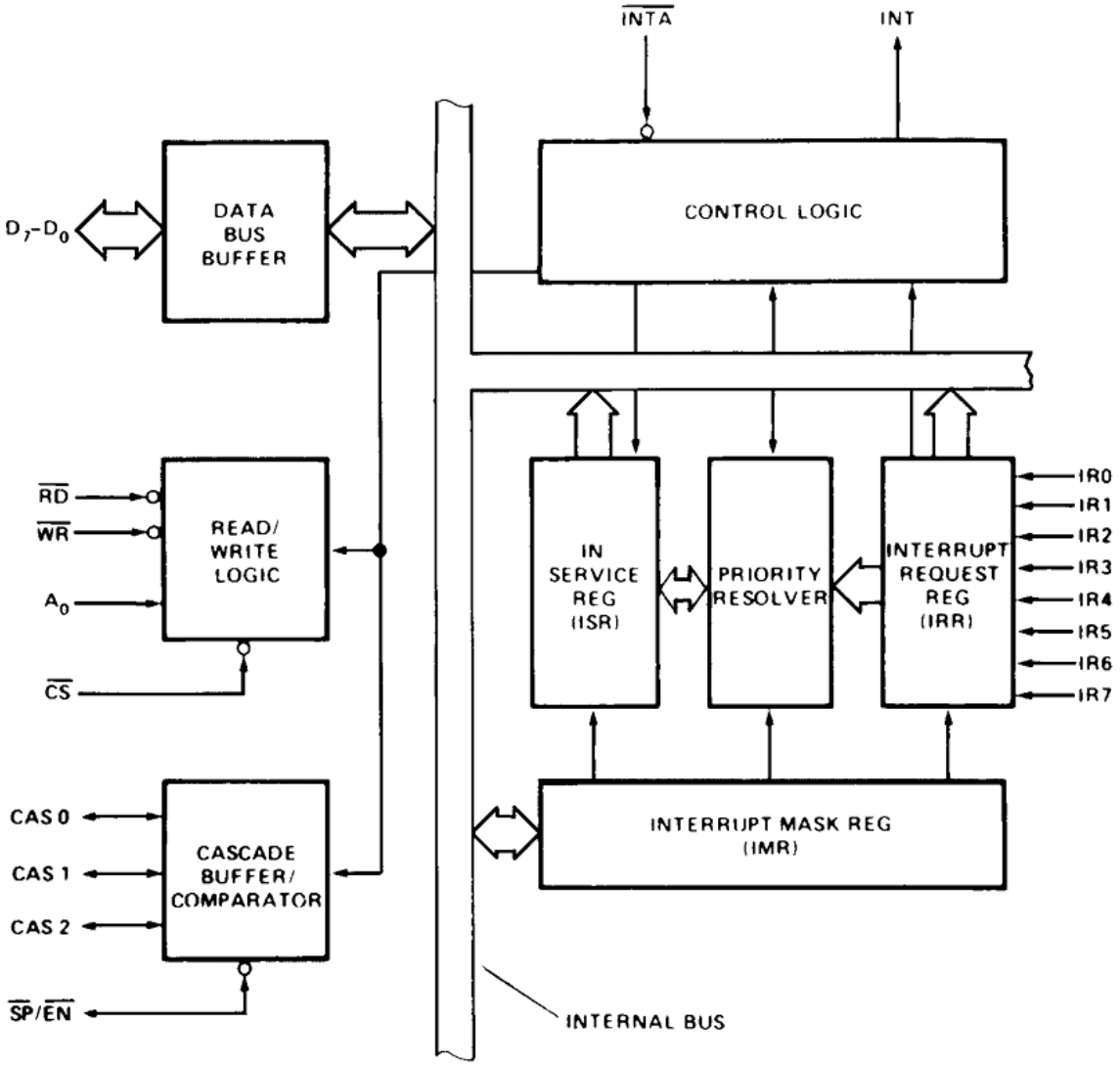
Logical Model of PIC Behavior (cont)

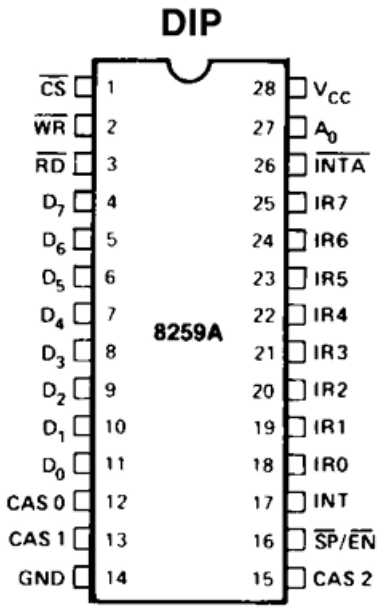
- When a device raises an interrupt:
  - check if priority is **higher** than those of in-service interrupts.
  - if not: do nothing
    - *what happens if we wait too long?*
    - buffer overflow, interrupt storm.
  - if so:
    - report the highest-priority raised to the processor.
    - mark that device as being in service.

Logical Model of PIC Behavior (cont.)

- When processor reports EOI (end of interrupt) for some interrupt.
  - remove the interrupt from the in-service mask.
  - check for raised interrupt lines.
    - that should be reported to processor.
  - could also have automatic EOI (not used by Linux).

8259A
Programmable Interrupt Controller (PIC)





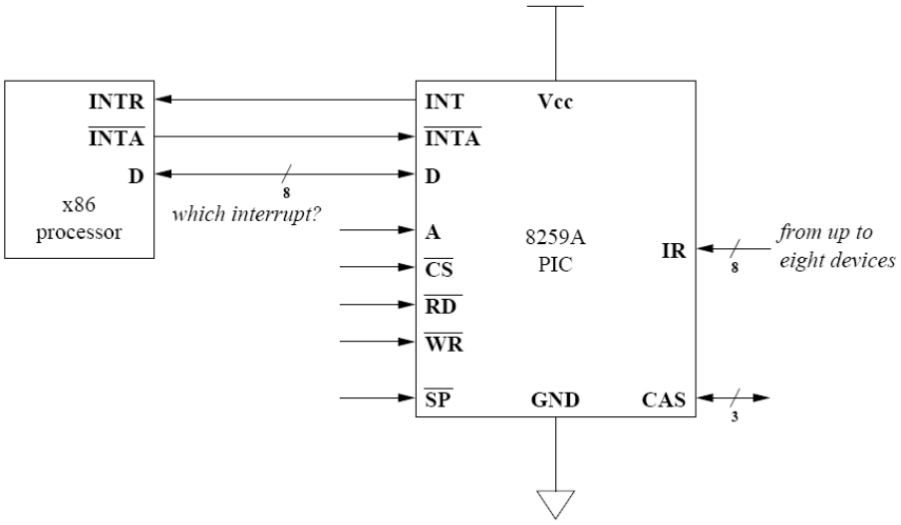
Logical Model of PIC Behavior (cont.)

- Protocol for reporting interrupts:
  - PIC raises INTR
  - processor strobes INTA' (active low) repeatedly
    - creates cycles for PIC to write vector to data bus
    - (must follow spec timing! PIC is not infinitely fast!)
  - processor sends EOI with specific combinations of A & D inputs (A is from address bus, D is from data bus)
    - more details in future lecture
  - Check the datasheet!

8259A Operation in 8086 Mode

The events occur as follows in an MCS-80/85 system:

1. One or more of the INTERRUPT REQUEST lines (IR7–0) are raised high, setting the corresponding IRR bit(s).
4. Upon receiving an $\overline{\text{INTA}}$ from the CPU group, the highest priority ISR bit is set and the corresponding IRR bit is reset. The 8259A does not drive the Data Bus during this cycle.

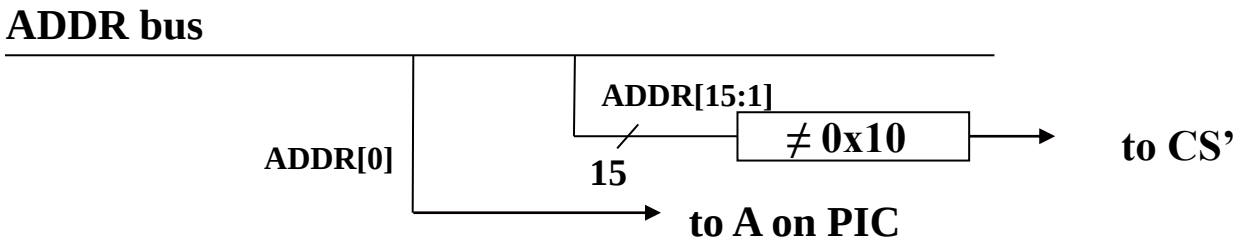


Logical Model of PIC Behavior (cont.)

- What about A, CS', RD', and WR' ?
  - A = address match for ports.
  - CS' = chip select (does processor want PIC to read/write?).
  - RD' and WR' defined from processor's point of view:
    - RD' = processor will read data (vector #) from PIC.
    - WR' = processor will write data (command, EOI) to PIC.

Logical Model of PIC Behavior (cont.)

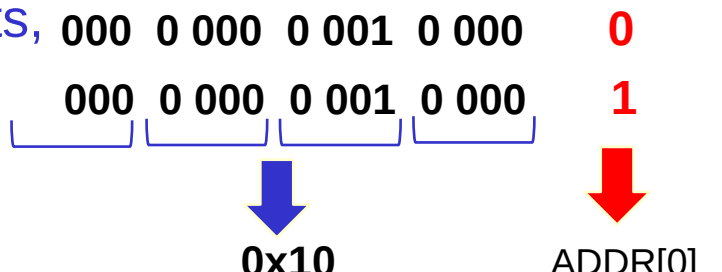
- Map to ports 0x20 & 0x21:
  - given ADDR bus.
  - logic to form A and CS' inputs to PIC.



- Remember:
  - the PIC is asynchronous!
  - all interactions have timing constraints,
  - see specifications for details.

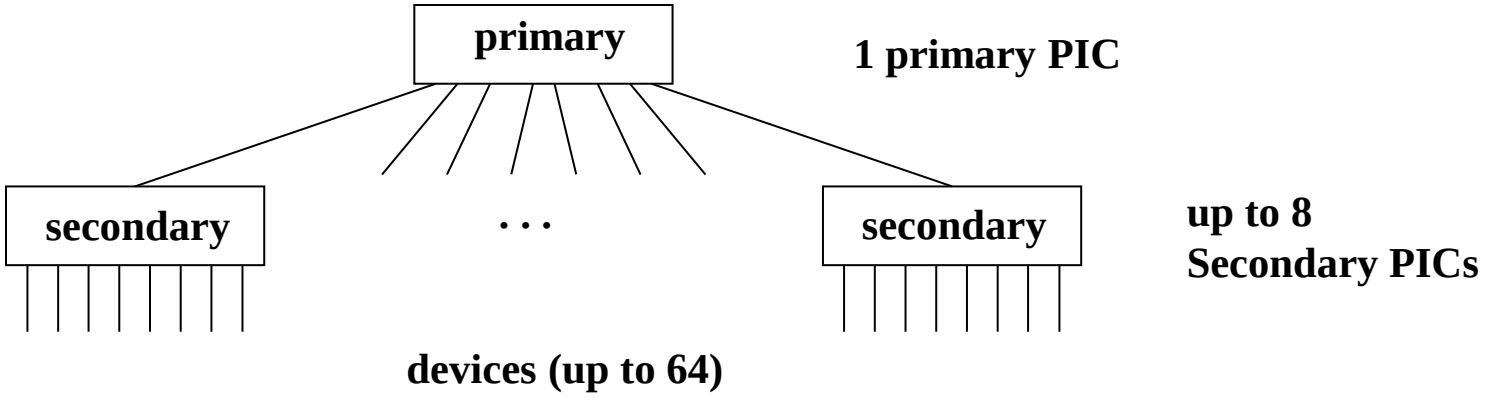
0x20 => 0000 0000 0010 000 0

0x21 => 0000 0000 0010 000 1

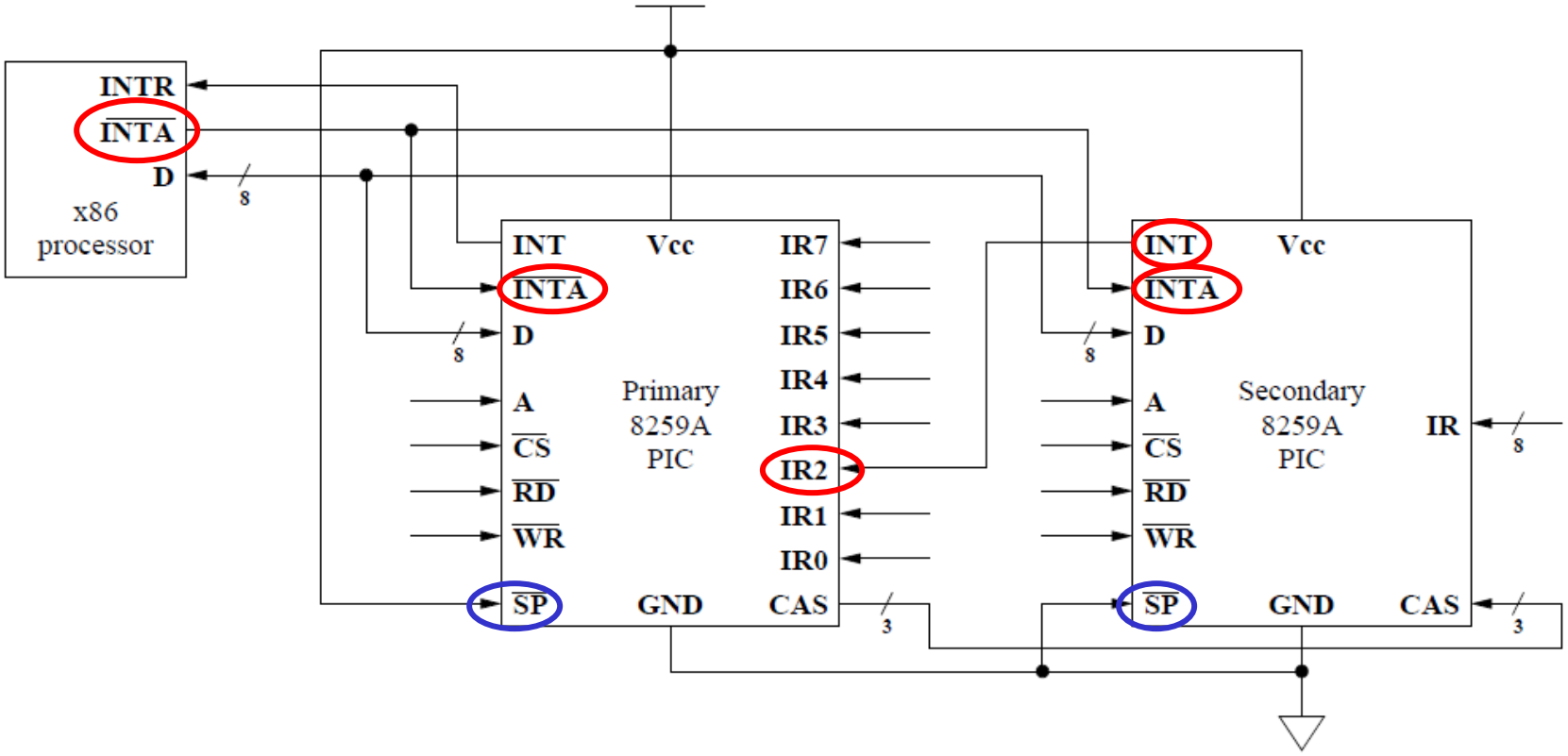


Logical Model of PIC Behavior (cont.)

- What about SP' & CAS?
- Are eight devices enough? No? What then?
  - hook 2, 3, or 4 PICs to processor?
    [same problem as before!]
  - design a big PIC?
    [waste of transistors; not cheap in 8259A era]
  - design several PICs?
    [production costs]
- Better answer: **Cascade**



Cascade Configuration of PICs



PIC (cont.)

- Previous figure showed x86 configuration of two 8259A's
  - primary 8259A mapped to ports 0x20 & 0x21.
  - secondary 8259A mapped to ports 0xA0 & 0xA1.
  - secondary PIC connects to IR2 on primary PIC.

- Question:
  - prioritization on 8259A: 0 is high, 7 is low.
  - what is prioritization across all 15 pins in x86 layout?
  - (highest) P0…P1…S0…S7…P3…P7 (lowest)

PIC (cont.)

- In Linux (initialization code to be seen next lecture)
  - primary PIC IR's mapped to vector #'s 0x20 – 0x27.
  - secondary PIC IR's mapped to vector #'s 0x28 – 0x2F.
  - remember the IDT?
- Vector byte from PIC:
  - put on data line.

Content of Interrupt Vector Byte for 8086 System Mode

	D7	D6	D5	D4	D3	D2	D1	D0
IR7	T7	T6	T5	T4	T3	1	1	1
IR6	T7	T6	T5	T4	T3	1	1	0
IR5	T7	T6	T5	T4	T3	1	0	1
IR4	T7	T6	T5	T4	T3	1	0	0
IR3	T7	T6	T5	T4	T3	0	1	1
IR2	T7	T6	T5	T4	T3	0	1	0
IR1	T7	T6	T5	T4	T3	0	0	1
IR0	T7	T6	T5	T4	T3	0	0	0

Interrupt Descriptor Table (IDT)

0x00–0x1F defined by Intel	0x00	division error	
	⋮		
	0x02	NMI (non-maskable interrupt)	
	0x03	breakpoint (used by KGDB)	
	0x04	overflow	
	⋮		
	0x0B	segment not present	
	0x0C	stack segment fault	
	0x0D	general protection fault	
	0x0E	page fault	
	⋮		
0x20–0x27 primary 8259 PIC	0x20	IRQ0 — timer chip	example of possible settings
	0x21	IRQ1 — keyboard	
	0x22	IRQ2 — (cascade to secondary)	
	0x23	IRQ3	
	0x24	IRQ4 — serial port (KGDB)	
	0x25	IRQ5	
	0x26	IRQ6	
	0x27	IRQ7	
0x28–0x2F secondary 8259 PIC	0x28	IRQ8 — real time clock	
	0x29	IRQ9	
	0x2A	IRQ10	
	0x2B	IRQ11 — eth0 (network)	
	0x2C	IRQ12 — PS/2 mouse	
	0x2D	IRQ13	
	0x2E	IRQ14 — ide0 (hard drive)	
	0x2F	IRQ15	
0x30–0x7F	⋮	APIC vectors available to device drivers	
0x80	0x80	system call vector (INT 0x80)	
0x81–0xEE	⋮	more APIC vectors available to device drivers	
0xEF	0xEF	local APIC timer	
0xF0–0xFF	⋮	symmetric multiprocessor (SMP) communication vectors	

Initialization Words

- 8259A accepts two types of command words:
  - **I**nitialization **C**ommand **W**ords (**ICW**)
  - **O**peration **C**ommand **W**ords (**OCW**)
- Initialization words:
  - Sent at beginning to start initialization sequence.
  - Any command with A = 0 (writing to 0x20) and D4 = 1 is recognized as ICW1.